

Situation: Parsing table conflict in LL(1)

Sol:-

An LL(1) parser is top down parser that reads the input from left to right and produces a left most derivation using one look ahead symbol. The parsing table is constructed using the first and follow sets.

first set: contains the terminals that can appear at the beginning of strings derived from a non terminal.

follow set: contains the terminals that can appear immediately after a non terminal.

while constructing the parsing table each production is placed in a table entry according to the first and follow sets.

A Parsing table conflict occurs when more than one production is entered into the same table cell for a non-terminal and a input symbol.

- left recursion
- left factoring problems.
- Ambiguous grammar.



## Decision

to make the grammar suitable for LL(1) parsing

Eliminate left recursion

Perform left factoring

Ensure first and follow sets do not overlap.

After these modifications each parsing table entry contains only one production resulting deterministic and conflict-free parsing.

## Q Scenario: incorrect parse tree generation

Sol: A syntax analyzer constructs a parse tree by applying grammar production according to the input tokens. The parse tree represents the syntactic structure of the expression.

For the expression:

$id + id - id$

The order of evaluation depends on the structure of the parse tree.

If the grammar does not define associativity the parser may generate the parse tree as:

$id + (id - id)$

instead of

$(id + id) - id$

Since addition and subtraction are left-associative the second interpretation is correct.

## Decision

modify the grammar to enforce left associativity



grammar  
 $C \rightarrow E + T \mid E - T \mid T$   
 $T \rightarrow id$

This grammar assigns expressions and evaluated left to right producing the correct parse tree and correct evaluation.

### 3. scenario of invalid expression Handling answer:

The lexical analyzer converts the input  
 $a + b$

into valid tokens

Identifier (a)

Operator (+)

Operator (\*)

Identifier (b)

After the '+' operator the grammar expects an operand (Identifier or expression) instead another operator '\*' appears.

Therefore the parser cannot construct a valid parse tree and reports a syntax error.

in predictive parsing (LL parser):

- The parser checks the parsing table.
- Since no valid production exists it reports an error.

in shift-reduce parsing (LR parser):

- The parser checks the Action Table.



without recovery parsing stops runtime directly.

### Decision:

to improve parsing:

- use panic mode recovery.
- use phrase-level recovery.
- use synchronizing tokens based on follow sets.
- display meaningful error message.

These techniques help the parser continue parsing the remaining and improve compiler error reporting.

### 4 Scenario: Ambiguity in conditional statements

Sol:

Given grammar:

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow a$

This

grammar is ambiguous because the parser cannot determine to which of statements the  $\text{else}$  belongs.

This ambiguity to which of statements the  $\text{else}$  belongs.

Both productions being with the same prefix  $E \text{ then } S$ .



## Scenario: Operator precedence conflict

Given grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

This grammar is ambiguous because it does not define operator precedence or associativity.

for the expression.

$$id + id * id$$

Two parse trees are possible.

1.  $(id + id) * id$

2.  $id + (id * id)$

the correct evaluation is:

$$id + (id * id).$$

because multiplication has higher precedence addition.

without precedence



During parsing the parser cannot decide whether to:

Reduce using  $S \rightarrow \text{if } E \text{ then } S$

continue using  $S \rightarrow \text{if } E \text{ then } S \text{ else } S$

this creates parsing conflicts and multiple possible parse trees.

Decision:

Rewrite the grammar using matched and unmatched statements

$S \rightarrow M \mid U$

$M \rightarrow \text{if } E \text{ then } M \text{ else } M \mid a$

$U \rightarrow \text{if } E \text{ then } S$

$\mid \text{if } E \text{ then } M \text{ else } U$

This grammar removes ambiguity.

Another common solution used by programming languages is:

Associate every class with the nearest unmatched  $\text{if}$ .

This completely resolves the dangling else problem.